

CloudShield AI

29 — Cloud Integration Specification

Version 1.0 | Product-Level Cloud Provider Integration Specification

CloudShield AI / CloudSpartanX

Document purpose: Define how CloudShield AI connects to, discovers, normalizes, analyzes, verifies, and—when explicitly governed—changes customer cloud infrastructure. AWS is the primary MVP provider, while provider-specific implementation is isolated behind a stable cloud integration abstraction.

Canonical principle: CloudShield AI must understand customer infrastructure without coupling its product domain to a single cloud SDK. Provider adapters translate provider APIs into canonical CloudShield resources, observations, relationships, capabilities, and execution results.

1. Purpose and Scope

This specification defines the cloud integration layer between CloudShield AI product/domain services and external cloud providers. It complements document 27, which defines CloudShield AI's own AWS hosting infrastructure, and document 28, which defines the product API.

- Define the provider-neutral cloud integration abstraction.
- Define AWS integration as the MVP provider.
- Define customer account, tenant, region, and resource boundaries.
- Define authentication, authorization, role assumption, and credential handling.
- Define discovery, inventory, normalization, configuration observation, and reconciliation.
- Define canonical provider-resource identity and asset mapping.
- Define dependency and relationship discovery.
- Define security-analysis data acquisition and provider capability detection.
- Define governed cloud mutation interfaces used only after DAIL and governance gates.
- Define provider error handling, retries, rate limits, pagination, eventual consistency, and reconciliation.
- Define testing, observability, security, and future multi-cloud extensibility.

2. Integration Architecture

The Cloud Integration Layer is a bounded adapter boundary. Domain services depend on canonical CloudShield interfaces; provider adapters depend on AWS/provider SDKs and APIs.

CloudShield AI Domain → Cloud Integration Interface → Provider Adapter → Provider API/SDK → Customer Cloud

Layer	Responsibility	Provider-Specific?
Domain	Assets, findings, dependencies, candidates, lifecycle, trusted state.	No
Cloud Integration Interface	Canonical discovery/read/execute contracts.	No
Provider Adapter	Translate canonical requests to	Yes

Layer	Responsibility	Provider-Specific?
	provider operations.	
Credential/Session Manager	Obtain scoped provider sessions/credentials.	Partly
Provider API/SDK	Actual cloud control-plane operations.	Yes
Normalizer	Map provider responses into canonical objects.	No/adaptor-assisted
Reconciler	Resolve identity, state, pagination, partial/ambiguous outcomes.	No

3. Provider-Neutral Contract

Capability	Canonical Operation
Connection	validate_connection()
Account/Scope	list_scopes(), get_scope_metadata()
Discovery	discover_resources(), discover_resource()
Configuration	get_configuration(), get_configuration_batch()
Relationships	discover_relationships()
Capabilities	get_capabilities()
Health	get_provider_health()/validate_access()
Execution	validate_change(), execute_change(), get_operation_status()
Reconciliation	reconcile_operation(), reconcile_resource()
Evidence	return source/provenance metadata for every material observation.

Provider adapters may implement additional capabilities, but the product must not require provider-specific behavior for core domain semantics.

4. AWS MVP Scope

AWS Area	MVP Coverage
IAM/STS	Connection validation and scoped role assumption.
EC2	Instances, security groups, networking metadata, volumes and related configuration as required.
VPC	VPCs, subnets, route/security metadata and relationships.
S3	Buckets, encryption, public-access configuration, policy/configuration metadata.
RDS	DB instances/configuration/security metadata.
Lambda	Functions, runtime/configuration/security-relevant settings.
ECS	Clusters/services/tasks/configuration where required.
CloudWatch	Relevant operational signals where needed for analysis.
IAM	Selected identity/policy metadata required for security analysis.
CloudTrail	Relevant audit/configuration evidence where available.
Additional services	Added through adapter modules when product requirements justify them.

The MVP service set should be expanded based on actual CloudShield AI security rules and customer use cases rather than attempting exhaustive AWS coverage on day one.

5. Customer Cloud Connection Model

Object	Definition
Tenant	CloudShield AI authorization boundary.
CloudConnection	Registered connection to a customer cloud account/environment.
CloudAccount	Provider account/project/subscription represented by the connection.
CloudScope	Region, account, resource group, or equivalent operation boundary.
ProviderRole	Customer-side role assumed by CloudShield AI.
CredentialReference	Secure reference to connection credentials or role configuration.
ConnectionPolicy	Product rules controlling permitted provider operations.
CapabilitySet	Provider operations/resources supported by this connection.

- A connection belongs to exactly one tenant.
- A connection may expose one or more cloud scopes.
- Connection secrets are never returned through the API.
- Connection identity and access configuration are separate from discovered assets.
- Disabling a connection blocks new provider operations but preserves historical evidence according to retention policy.

6. AWS Authentication and Role Assumption

For AWS customer environments, the preferred pattern is a customer-side IAM role that CloudShield AI assumes through a controlled trust relationship. Long-lived customer access keys should not be the default architecture.

- Use short-lived STS credentials for runtime operations.
- Separate read/discovery and mutation/promotion roles.
- Restrict trust policies to the intended CloudShield AI principal.
- Use external identifiers or equivalent anti-confusion controls where appropriate.
- Limit role session duration and permissions.
- Record connection and role references in audit metadata without logging credentials.
- Rotate or revoke access by disabling the connection or changing the customer-side trust policy.

Role	Purpose	Mutation
DiscoveryRole	Inventory and configuration reads.	No
AnalysisRole	Optional specialized read access.	No
VerificationRole	Read APIs needed to validate candidate effects.	No or minimal, explicitly scoped
PromotionRole	Execute approved change operations.	Yes, tightly scoped

7. Credential and Secret Boundary

- Customer credentials are stored/referenced only through approved secret/identity mechanisms.
- Application code must not persist raw temporary credentials in domain records.
- Provider credentials never cross into the browser.
- AI workers do not receive customer-cloud mutation credentials.
- Logs, traces, errors, metrics, and evidence must redact credentials and authorization tokens.
- Connection tests return safe diagnostics rather than provider credential material.
- Credential expiry must trigger re-acquisition or a controlled connection failure.

8. Cloud Discovery Pipeline

Discovery converts provider-native infrastructure into canonical CloudShield observations and assets.

1. Validate tenant and CloudConnection authorization.
2. Acquire a scoped provider session.
3. Enumerate supported resources using provider pagination.
4. Resolve canonical provider resource identifiers.
5. Normalize resource metadata into CloudShield Asset objects.
6. Record observations with source, timestamp, scope, and provenance.
7. Discover relevant configuration and security metadata.
8. Discover supported relationships/dependencies.
9. Persist observations and reconciliation results.
10. Emit discovery completion/partial/failure state and evidence.

Discovery must be incremental and resumable where provider APIs and workload scale require it.

9. Resource Identity Normalization

CloudShield AI must maintain stable identity independent of display names and mutable provider metadata.

Identity Field	Purpose
provider	AWS in MVP.
account_id	Customer cloud account identifier.
region	Provider region where applicable.
resource_type	Canonicalized provider resource type.
provider_resource_id	Stable provider identifier/ARN where applicable.
canonical_asset_id	CloudShield AI stable asset identifier.
identity_version	Version of identity normalization logic.

- Display names/tags are attributes, never primary identity.
- Provider identifiers must be normalized consistently.
- Ambiguous identity matches create a conflict rather than an unsafe merge.
- Deleted/recreated resources are distinct when provider identity semantics indicate a new resource.
- Identity reconciliation is auditable.

10. Canonical Asset Mapping

Provider Data	CloudShield Object
AWS ARN/resource ID	ResourceIdentifier + Asset

Provider Data	CloudShield Object
Tags/name	Asset attributes
Provider region	CloudScope
Configuration response	Observation/ConfigurationSnapshot
Cloud API timestamp	Observation provenance
Related resource reference	DependencyEdge candidate
API capability/permission error	Discovery diagnostic/evidence

Normalization should preserve enough provider-specific metadata to support evidence and future adapter improvements without contaminating the canonical domain model.

11. Configuration Observation

- Provider configuration is stored as normalized, typed data where possible.
- Raw provider responses may be retained as evidence artifacts when necessary.
- Observation timestamps are mandatory for security-relevant facts.
- Configuration snapshots are immutable historical records.
- Changes between snapshots produce ConfigurationDiff objects.
- Provider eventual consistency is represented explicitly.
- An unavailable field is not equivalent to a secure/default field.

12. Discovery Freshness

Status	Meaning
FRESH	Within configured freshness policy.
STALE	Older than expected but still retained.
UNKNOWN	Freshness cannot be established.
PARTIAL	Discovery completed only for a subset.
INVALIDATED	Previously derived state is no longer trustworthy.
FAILED	Required discovery operation did not complete.

Freshness must influence analysis and promotion. A stale or partial observation cannot silently satisfy a freshness-sensitive safety gate.

13. Pagination and Rate Limits

- Provider pagination is handled inside the adapter; domain services should not need provider page-token semantics.
- Adapters must preserve progress checkpoints for large discovery jobs where practical.
- Provider rate limits trigger bounded exponential backoff with jitter.
- Retries must honor provider retry guidance.
- Repeated rate limiting must degrade the job explicitly rather than creating incomplete false-clean state.
- Discovery concurrency is bounded per provider/account/region.
- Promotion operations use much stricter concurrency controls than discovery.

14. Retry and Timeout Policy

Operation	Retry Strategy
-----------	----------------

Operation	Retry Strategy
Read/discovery	Retry transient provider errors with bounded backoff.
Configuration read	Retry transient errors; preserve partial status.
AI-adjacent context read	Fail/partial explicitly if authoritative context cannot be obtained.
Verification read	Retry transient errors, then block verification if required evidence is unavailable.
Mutation	Idempotent provider operation where supported; never blindly retry ambiguous side effects.
Promotion status query	Retry and reconcile until outcome is authoritative or marked ambiguous.

A timeout is not proof that an external mutation failed. For side-effecting operations, the integration layer must enter reconciliation when the outcome cannot be determined safely.

15. AWS API Error Mapping

Provider Condition	CloudShield Result
AccessDenied	PERMISSION_DENIED + evidence/diagnostic.
Throttling	RETRYABLE_PROVIDER_ERROR / backoff.
NotFound	RESOURCE_NOT_FOUND if authoritative.
Validation error	INVALID_PROVIDER_REQUEST.
Service unavailable	PROVIDER_UNAVAILABLE.
Timeout/network failure	PROVIDER_TIMEOUT.
Ambiguous mutation outcome	RECONCILIATION_REQUIRED.
Unsupported operation	CAPABILITY_UNSUPPORTED.

Provider error details are logged securely for operators but mapped to stable domain/API semantics for product clients.

16. Capability Discovery

CloudShield AI must know what the connection can actually support before executing analysis or remediation.

Capability	Example
resource_discovery	Can enumerate S3 buckets.
configuration_read	Can inspect S3 public-access settings.
relationship_read	Can resolve relevant dependencies.
security_analysis	Required inputs for a rule are available.
verification	Can observe expected post-change state.
mutation	A specific change operation is permitted.
evidence	Required provider evidence can be collected.

Unsupported capability must produce an explicit unsupported/unknown result, not an implicit secure result.

17. Dependency and Relationship Discovery

The cloud integration layer supplies provider relationships that feed the canonical dependency graph.

- Discover explicit provider relationships where available.

- Derive safe relationships from authoritative configuration only when derivation rules are versioned.
- Attach provenance to safety-relevant dependency edges.
- Distinguish observed edges from inferred edges.
- Assign confidence and freshness metadata.
- Do not manufacture relationships from names or weak heuristics.
- Unknown relationships remain unknown and can increase impact uncertainty.

Edge Provenance	Meaning
PROVIDER_OBSERVED	Directly supplied by provider API/configuration.
CONFIG_DERIVED	Deterministically derived from authoritative configuration.
INFERRED	Higher uncertainty; not sufficient alone for safety-critical assumptions.
UNKNOWN	Relationship could not be established.

18. Security Analysis Integration

Security rules consume canonical observations rather than directly calling provider SDKs.

Provider API → Adapter → Observation/ConfigurationSnapshot → Security Rule Engine → Finding → RiskAssessment

- Rules declare required data/capabilities.
- Missing required provider data creates UNKNOWN/INCOMPLETE evaluation where appropriate.
- Rule versions are recorded with findings.
- Provider-specific quirks are normalized in the adapter.
- Security logic remains portable across future providers.

19. Cloud Change Execution Boundary

Cloud mutation is a separate capability from discovery. The adapter exposes only explicitly supported operations and receives a fully specified, authorized change command.

Precondition	Required Before Mutation
Authenticated connection	Yes
Authorized scope	Yes
Supported capability	Yes
Exact candidate version	Yes
Impact analysis	Yes, where required
DAIL verification	Yes for governed changes
Policy decision	Yes
Approval	When policy requires
Final precondition recheck	Yes
Audit/evidence context	Yes

The cloud adapter itself must reject mutation commands that lack required authorization/context even if an upstream service mistakenly attempts to issue them.

20. Change Command Contract

Field	Purpose
candidate_version_id	Exact immutable candidate version.
cloud_connection_id	Target customer connection.
scope_id	Authorized account/region/resource scope.
operation	Allow-listed provider operation.
target_resource_id	Canonical CloudShield asset/resource.
provider_target_id	Resolved provider identifier.
desired_change	Normalized change specification.
preconditions	Expected current state.
policy_context	Policy/version references.
verification_context	Required verification contract.
correlation_id	End-to-end traceability.

The adapter must resolve and validate provider target identity rather than trusting arbitrary provider IDs supplied by an untrusted client.

21. Precondition Validation

- Verify the connection is active.
- Verify the target resource still resolves to the expected provider identity.
- Re-read critical current state immediately before mutation where required.
- Verify candidate version has not been superseded or invalidated.
- Verify dependency/impact assumptions that are safety-critical.
- Verify required IAM permissions/capabilities.
- Verify policy and approval context remain valid.
- Abort rather than execute if critical preconditions are missing or ambiguous.

22. Promotion Execution

11. Promotion service submits a governed ChangeCommand.
12. Cloud Integration validates connection, scope, capability, and target identity.
13. Critical current state is re-read.
14. Provider-specific operation is constructed from normalized desired change.
15. Provider mutation executes using PromotionRole.
16. Operation result is captured.
17. Post-change observation/verification data is collected.
18. Evidence and audit records are written.
19. Promotion becomes SUCCEEDED only when authoritative success criteria are met.
20. Trusted State advances only after the governed promotion contract is satisfied.

23. Ambiguous External Outcomes

External mutation calls can time out after the provider has accepted the request. CloudShield AI must distinguish failure from unknown outcome.

- Mark the operation RECONCILIATION_REQUIRED.
- Do not automatically retry a non-idempotent mutation.

- Query provider operation/resource state using authoritative APIs.
- Compare actual state to the exact candidate desired state.
- Record reconciliation evidence.
- Only then transition to succeeded, failed, or a governance-required state.
- Do not mark trusted state based solely on client/network response.

24. Eventual Consistency

- Provider APIs may return stale or transitional state immediately after changes.
- Verification checks must define acceptable convergence windows.
- Observation timestamps and provider operation identifiers should be preserved.
- Transient mismatch during an explicitly allowed convergence window is not automatically a permanent failure.
- After the convergence window, unresolved mismatch blocks promotion/trusted-state advancement.
- Reconciliation must use authoritative provider reads.

25. Cloud Integration Security Model

Boundary	Control
Tenant → Connection	Server-side authorization.
CloudShield → Customer AWS	Explicit trust relationship and scoped role.
Discovery → Mutation	Separate identities and contracts.
AI → Cloud	No direct mutation authority.
DAIL → Cloud	Dedicated verification/promotion identities.
Provider SDK → Domain	Adapter isolation.
Credential → Workload	Short-lived/scoped secret access.
Provider response → Domain	Validation/normalization before persistence.

26. Tenant and Scope Isolation

- Every provider operation carries an authorized tenant and CloudConnection context.
- Adapters cannot select a connection solely from a client-supplied provider account ID.
- Cross-tenant resource references are rejected.
- Scope checks occur before provider calls where possible and again before mutation.
- Connection metadata is isolated from discovered resource data by authorization.
- Background workers must revalidate tenant/scope context because queue payloads are untrusted.

27. Observability

Signal	Required Information
Connection	Connection ID, provider, account/scope, success/failure class.
Discovery	Job ID, service, resource type, counts, duration, partial/failure status.
Provider call	Provider operation class, latency, retry count, safe error class.
Rate limit	Provider/account/operation class and backoff.
Mutation	Candidate version, target, operation class, outcome.
Reconciliation	Operation ID, state, evidence, final result.

Signal	Required Information
Capability	Supported/unsupported/permission-limited state.

Raw request/response payloads should not be logged by default. Sensitive provider responses are retained only when required as protected evidence.

28. Evidence and Provenance

- Every material cloud observation carries source/provider and observation time.
- Configuration snapshots identify the connection/scope used.
- Mutation evidence records operation identity, target, preconditions, result, and post-change observation.
- Evidence artifacts are immutable or append-only according to the evidence specification.
- Provider API errors that affect security conclusions are preserved as diagnostic evidence.
- Evidence access is authorized and auditable.

29. Caching

Caching is allowed for low-risk, non-authoritative provider metadata but must never hide stale safety-critical state.

Data	Caching Rule
Provider capability metadata	Cache with explicit TTL.
Static region/service metadata	Longer TTL acceptable.
Asset display metadata	Short TTL acceptable.
Security configuration	Prefer authoritative reads or tightly bounded freshness.
Pre-promotion state	Do not rely on stale cache; re-read.
Verification state	Use authoritative reads as required by verification contract.

30. Provider Adapter Structure

Module	Responsibility
provider_client	SDK/API session management.
auth	Role/session acquisition.
discovery	Resource enumeration.
normalizer	Canonical mapping.
configuration	Configuration reads.
relationships	Dependency extraction.
capabilities	Feature/permission detection.
execution	Allow-listed mutations.
reconciliation	Ambiguous operation resolution.
errors	Stable error mapping.
tests	Contract and provider integration tests.

31. Recommended Code Boundary

Application/domain code should depend on interfaces similar to:

- CloudConnectionProvider

- CloudResourceDiscovery
- CloudConfigurationReader
- CloudRelationshipReader
- CloudCapabilityResolver
- CloudChangeValidator
- CloudChangeExecutor
- CloudOperationReconciler

AWS-specific implementations such as AWSCloudProviderAdapter implement these interfaces. The domain layer must not import AWS SDK types directly.

32. Testing Strategy

Test Class	Coverage
Contract tests	Canonical adapter interface behavior.
AWS adapter unit tests	Normalization, errors, pagination, capability mapping.
Provider integration tests	Real or controlled AWS test account.
IAM tests	Read role versus promotion role boundaries.
Tenant tests	Connection/scope isolation.
Discovery tests	Inventory completeness and identity reconciliation.
Mutation tests	Allow-listed changes only.
Failure tests	Timeout, throttling, access denied, not found.
Reconciliation tests	Ambiguous external outcomes.
Eventual consistency tests	Convergence and verification windows.
Security tests	Credential leakage, privilege escalation, SSRF/provider endpoint abuse.

33. Local Development and Mock Provider

- Provide a mock cloud adapter implementing the canonical interface.
- Unit tests should run without live AWS credentials.
- Mock fixtures should include successful, partial, stale, denied, throttled, and ambiguous outcomes.
- Development environments should use synthetic resource identifiers.
- Provider contract tests should run separately against controlled AWS test infrastructure.
- Never require developer laptops to contain production customer credentials.

34. AWS Test Account Strategy

A dedicated non-production AWS account/environment should be used for integration tests involving actual provider APIs.

- Use ephemeral or tightly scoped test resources.
- Apply least-privilege roles mirroring production permissions.
- Include representative networking, storage, compute, database, IAM, and logging resources.
- Test both discovery and controlled mutation paths.
- Destroy ephemeral resources after tests where safe.
- Keep production accounts completely outside automated integration-test mutation paths.

35. Performance and Scale

Workload	Design Requirement
Large inventory	Paginated/batched discovery and checkpointing.
Many regions	Bounded parallelism.
Many accounts	Per-connection concurrency controls.
Provider throttling	Adaptive backoff.
Dependency discovery	Incremental processing.
Security analysis	Decoupled from provider request latency via jobs.
Promotion	Low concurrency, high correctness.

The integration layer must optimize for correctness and provider safety before raw throughput, particularly for mutation operations.

36. Reliability Requirements

- Provider outages must not corrupt canonical domain state.
- Transient failures are retried only when safe.
- Persistent failures are represented explicitly.
- Discovery is resumable where practical.
- Mutation operations use idempotency/reconciliation semantics.
- Provider-side state is authoritative for post-change verification.
- Worker restarts must not create duplicate unsafe mutations.
- Connection health can degrade independently from historical observations.

37. Cloud Provider Expansion

Future providers such as Azure or Google Cloud should implement the same canonical capabilities without forcing the domain model to become provider-specific.

Extension Rule	Requirement
Identity	Map provider identifiers to stable CloudShield assets.
Scopes	Map subscription/project/account/region semantics.
Discovery	Return canonical observations.
Relationships	Return provenance and confidence.
Capabilities	Declare supported operations.
Mutation	Use explicit allow-listed change contracts.
Errors	Map to stable CloudShield error classes.
Verification	Provide authoritative post-change reads.

Provider differences that cannot be safely normalized should remain explicit capability differences rather than being hidden behind misleading equivalence.

38. Configuration and Feature Flags

- Enable/disable provider services through version-controlled configuration.
- Allow connection-level service discovery configuration.
- Allow policy-controlled mutation capabilities.
- Do not allow a client-side flag to bypass server-side authorization.

- Feature flags affecting promotion must be protected and audited.
- Provider API version/SDK changes are managed as controlled dependency changes.

39. Operational Runbooks

The implementation must maintain runbooks for:

- Expired/invalid customer role.
- AWS throttling or regional outage.
- Discovery partial failure.
- Unexpected resource identity conflict.
- Provider API schema/behavior change.
- Promotion timeout and reconciliation.
- Customer credential/role revocation.
- Incorrect permission configuration.
- Large-scale discovery backlog.
- Cloud integration security incident.

40. Security Acceptance Criteria

21. No customer cloud credential is exposed through the public API.
22. AI workers have no unrestricted customer-cloud mutation authority.
23. Discovery and promotion roles are separate.
24. Every customer-cloud operation is tenant and scope authorized.
25. Provider resource identity is canonicalized and auditable.
26. Provider errors cannot be interpreted as clean security state.
27. Ambiguous mutations enter reconciliation.
28. Critical pre-promotion state is re-read.
29. Provider SDK types do not leak into the core domain API.
30. Cloud integration logs do not contain secrets or access tokens.
31. Provider rate limiting cannot cause unsafe duplicate execution.
32. Cloud mutations are allow-listed and tied to exact candidate versions.

41. Functional Acceptance Criteria

33. A new AWS connection can be validated without exposing credentials.
34. Discovery can enumerate supported resources across configured scopes.
35. Provider resources map to stable CloudShield Asset IDs.
36. Configuration snapshots can be created and compared.
37. Relevant dependency relationships are persisted with provenance.
38. Security analysis can consume canonical observations without importing AWS SDK types.
39. Capability detection identifies unsupported or unauthorized operations.
40. A governed candidate can be translated into an AWS-specific change command.
41. Promotion rechecks target identity and critical current state.
42. Successful provider execution produces evidence and post-change observations.
43. Ambiguous provider outcomes can be reconciled without unsafe duplicate mutation.

42. MVP Implementation Sequence

44. Define canonical cloud integration interfaces.
45. Implement CloudConnection/session abstractions.
46. Implement AWS STS role-assumption adapter.
47. Implement AWS account/region scope discovery.
48. Implement core AWS resource discovery.
49. Implement normalization and canonical asset identity.
50. Implement configuration snapshots and provenance.
51. Implement dependency/relationship extraction.
52. Implement capability detection.
53. Implement provider error mapping and retry policies.
54. Implement controlled change validation.
55. Implement promotion execution with dedicated role.
56. Implement post-change observation and reconciliation.
57. Add evidence/audit integration.
58. Build mock-provider and AWS test-account suites.
59. Load-test discovery and validate throttling behavior.

43. MVP Provider Coverage Strategy

CloudShield AI should prioritize provider coverage based on security value and dependency centrality.

Priority	Coverage
P0	IAM, VPC/networking, S3, EC2, RDS, Lambda, core account/region metadata.
P1	ECS, CloudTrail, CloudWatch, additional identity/network/security services.
P2	Broader AWS service inventory and specialized remediation operations.
Future	Azure/GCP adapters and multi-cloud normalization.

44. Traceability

Source	Cloud Integration Dependency
00 — Master System Specification	Cloud product boundary, AWS MVP, discovery-to-trust control loop.
01 — Software Requirements Specification	Cloud onboarding, discovery, security analysis, remediation, verification, reliability.
02 — System Architecture Specification	Cloud adapter boundary, workers, data stores, asynchronous processing.
03 — Functional Specification	Discovery, findings, risk, dependency, candidate, verification, promotion workflows.
04 — Product & UX Specification	Connection status, discovery jobs, stale/partial states, trusted-state presentation.
05 — Domain & Data Model	CloudConnection, Asset, Observation, Snapshot, DependencyEdge, Candidate, Evidence.
06 — State & Lifecycle	Discovery, verification, promotion, invalidation, reconciliation.
07 — Identity & Dependency	Stable provider identity, graph relationships, impact semantics.

Source	Cloud Integration Dependency
08 — Security & Threat Model	IAM, tenant isolation, credentials, network, provider boundary.
09 — Intelligence & AI Architecture	AI context from authoritative cloud observations; AI remains non-authoritative.
10 — LLM Integration	Provider boundary and failure handling for AI services.
11 — AI Safety & Guardrails	AI must not receive unrestricted cloud mutation authority.
12–26 — DAIL subsystem	Verification, promotion, evidence, testing, Terraform/AWS integration.
27 — AWS & Terraform Specification	CloudShield AI's own AWS hosting, IAM, networking, secrets, queues and observability.
28 — API Specification	CloudConnection, asset, dependency, candidate, verification and promotion API contracts.

45. Engineering Decisions

Decision	Rationale
Provider adapter boundary	Prevents AWS SDK coupling in product/domain code.
AWS first	Matches MVP infrastructure strategy and product focus.
Role assumption	Reduces long-lived credential exposure.
Separate discovery/promotion roles	Limits blast radius.
Canonical asset identity	Makes the domain provider-independent.
Explicit capability model	Avoids pretending unsupported operations are available.
Reconciliation for ambiguous mutations	Prevents unsafe duplicate changes.
Authoritative pre-promotion reads	Protects against stale context.
Canonical relationship provenance	Supports safe dependency and impact analysis.
Mock provider	Allows fast local/unit testing without cloud access.

46. Definition of Done

- Canonical cloud integration interfaces are implemented and documented.
- AWS is implemented as the production MVP provider adapter.
- Customer AWS access uses scoped, auditable identity/role assumption.
- Discovery supports configured accounts/regions and paginated provider APIs.
- Provider resources map to stable CloudShield asset identities.
- Configuration observations include provenance and freshness.
- Dependency relationships include provenance/confidence semantics.
- Provider capabilities and permission limitations are explicit.
- Provider errors are mapped to stable CloudShield error classes.
- Safe retry/backoff policies are implemented.
- Mutation operations are allow-listed and tied to exact candidate versions.
- Promotion uses a dedicated restricted identity.
- Ambiguous mutation outcomes enter reconciliation.
- Post-change authoritative observations support verification.
- Evidence/audit integration is complete.
- Mock-provider and AWS integration tests pass.

- Security acceptance criteria pass.
- Core interfaces can support a future second cloud provider without changing domain semantics.

47. Glossary

Term	Definition
Cloud Integration Layer	Bounded subsystem translating provider APIs into CloudShield domain semantics.
Provider Adapter	Provider-specific implementation of canonical cloud interfaces.
CloudConnection	Authorized connection from a tenant to a customer cloud environment.
CloudScope	Account/region/resource boundary within a connection.
Provider Resource ID	Native stable cloud identifier such as an AWS ARN.
Canonical Asset ID	Stable CloudShield identifier for an infrastructure asset.
Capability	Declared provider operation/resource support available to a connection.
Discovery	Process of enumerating and observing customer cloud infrastructure.
Reconciliation	Process of resolving ambiguous or divergent external state.
PromotionRole	Restricted identity used for governed cloud mutation.
Observation	Time-stamped fact obtained from a cloud provider.
Trusted State	State earned after governed, verified execution.